Report

Connor Emmons

02 March 2026

Notes

1 First Attempt at Fixing Hamiltonian - Final Velocity Free

1.1 Preliminary Implementation

Changed the constraint that the full state had to be the same at the beginning and the end to only having to maintain x and y at the end. Then, the value to minimize is:

$$\sqrt{(v_{xf} - v_{xi})^2 + (v_{yf} - v_{yi})^2} \tag{1.1}$$

The solution ran very quickly (∼0.1s) and the Hamiltonian was zero for every step; however, the orbit was incorrect. The solution is essetially the initial guess, as evidenced by the segmented nature of the orbit.

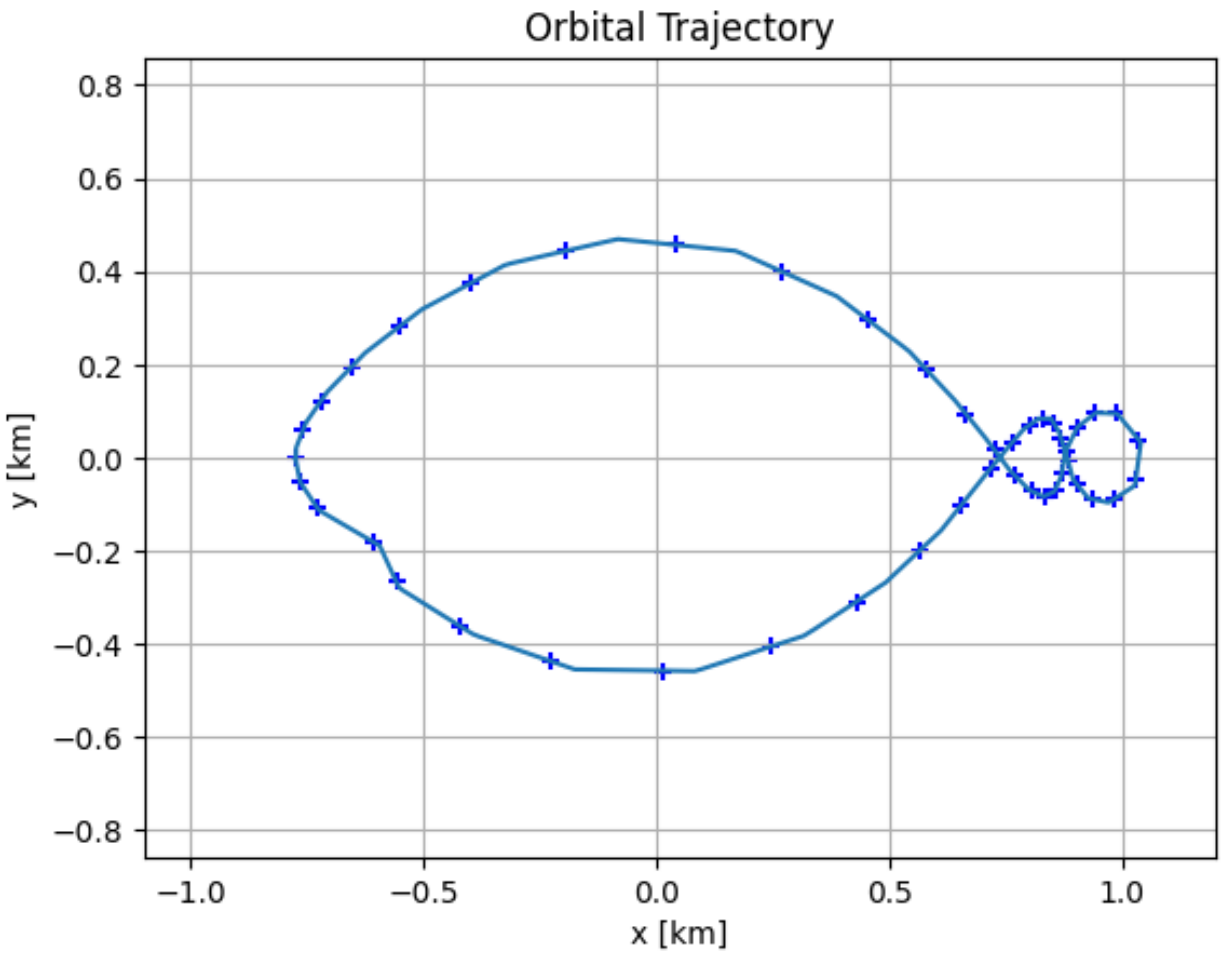

Figure 1: First orbit generated, note the segmented nature which reflects the manner that the initial guess is generated

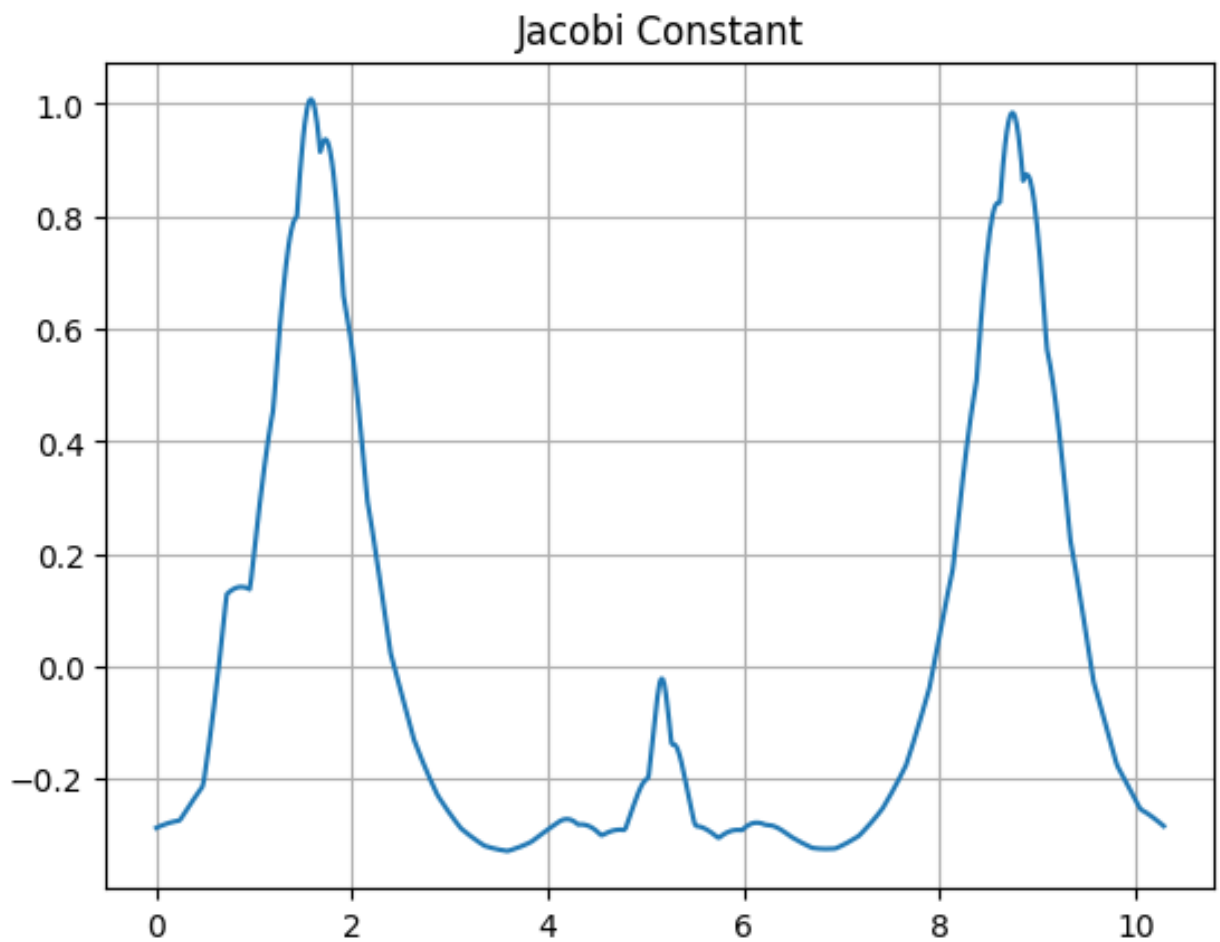

Figure 2: The corresponding Jacobi constant, no good

Removing the bounds on time had no effect.

1.2 Add Jacobi Constant to Cost

Change the value to minimize to:

$$\sqrt{(v_{xf} - v_{xi})^2 + (v_{yf} - v_{yi})^2} + W \int (C - C*)^2 \, dt \tag{1.2}$$

Here, $C*$ is the targeted Jacobi constant and W is an adjustable weight. The first run with $W = 1$ saw no change to the solution.

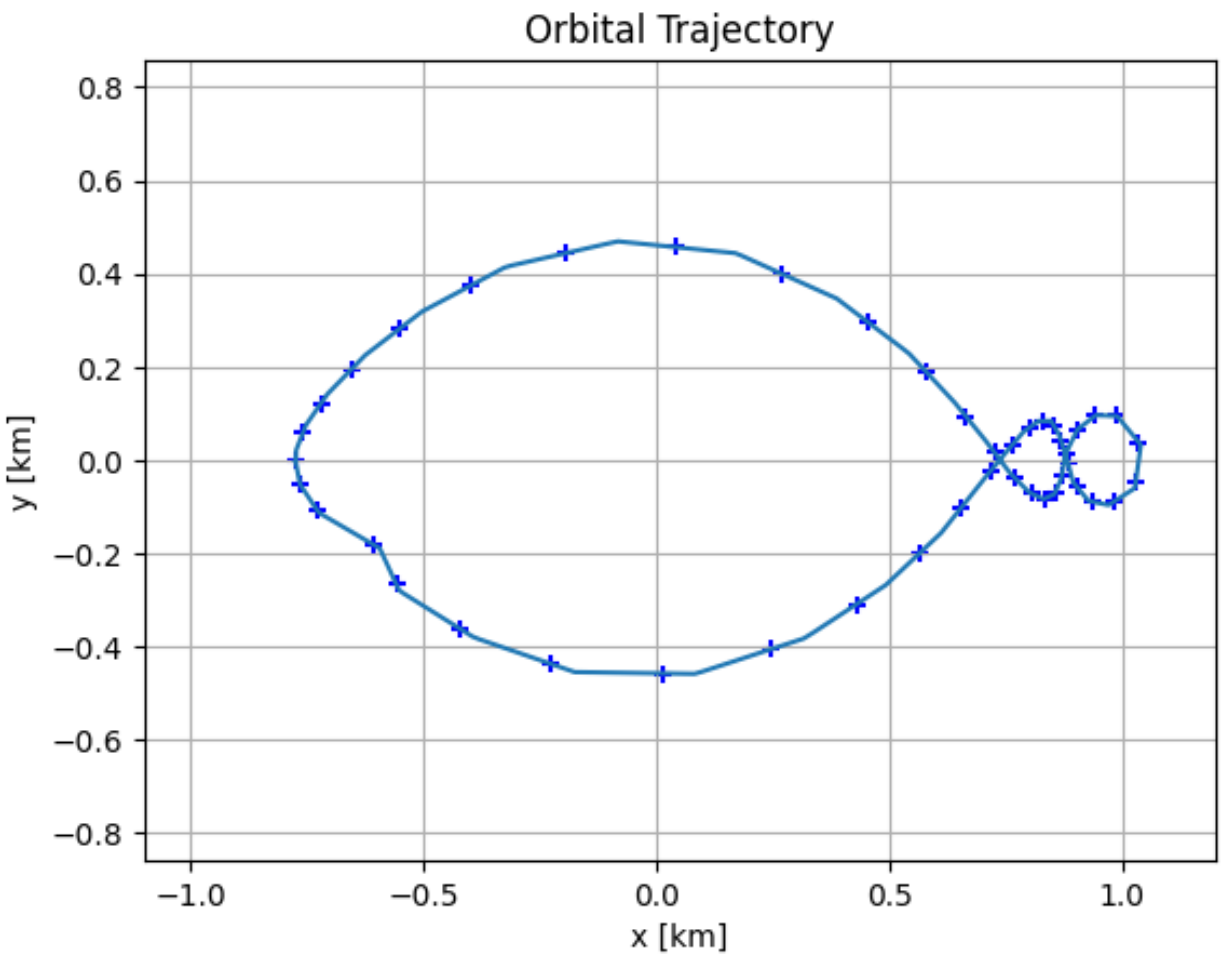

Figure 3: With W=1, no change

Increasing W to absurd numbers has no effect on the solution. In some way, this makes sense: the Jacobi constant is just a product of the dynamics of the system, which are already given. If YAPSS is ignoring the dynamics (which it is in the current solution), then adding in the Jacobi constant effectively does nothing.

1.3 Add Control

Now let YAPSS have control, specifically over the velocity. Now the goal is to minimize:

$$\sqrt{\left(v_{xf} - v_{xi}\right)^2 + \left(v_{yf} - v_{yi}\right)^2} + \int W_c \left(C - C*\right)^2 + W_u \left(v_x^2 + v_y^2\right) dt \tag{1.3}$$

This also resulted in no change to the solution. The orbit still mirrors the initial guess. Removing the Jacobi constant portion from the integral also had no effect. I am very confident that this implentation is garbage, and thus I have included a screenshot for future reference.

```python
C_value = 3.151175879508174
W_c = 1
W_u = 0.001
```

[46] ✓ 0.0s Python

```python
def objective(arg):
    xi, yi, vxi, vyi = arg.phase[0].initial_state
    xf, yf, vxf, vyf = arg.phase[0].final_state
    arg.objective = math.sqrt((vxf - vxi)**2 + (vyf - vyi)**2) + arg.phase[0].integral[0]

def continuous(arg):
    x, y, _, _ = arg.phase[0].state
    vx, vy = arg.phase[0].control
    r1 = math.sqrt((x + mu)**2 + y**2)
    r2 = math.sqrt((x - 1 + mu)**2 + y**2)
    ax = x - (1 - mu)*(x + mu)/r1**3 - mu*(x - 1 + mu)/r2**3 + 2*vy
    ay = y - (1 - mu)*y/r1**3 - mu*y/r2**3 - 2*vx

    C = -2*U(x, y) - (vx**2 + vy**2)

    arg.phase[0].dynamics = [vx, vy, ax, ay]
    arg.phase[0].integrand = W_c*(C - C_value)**2 + W_u*(vx**2 + vy**2)

def discrete(arg):
    xi, yi, vxi, vyi = arg.phase[0].initial_state
    xf, yf, vxf, vyf = arg.phase[0].final_state
    arg.discrete = [xf-xi, yf-yi]
```

[47] ✓ 0.0s Python

Figure 4: Code for control

1.4 Change Control to Acceleration

Instead of having YAPSS control the velocity, have YAPSS be able to modify the acceleration by adding ux and uy to the corresponding accelerations. This also had absolutely no change on the solution, regardless of the weighting.

2 New Idea - Velocity as Control

Reduce the state to just the position and let YAPSS have full control over the velocity components. This was a dumb idea, and the orbit reflects this.

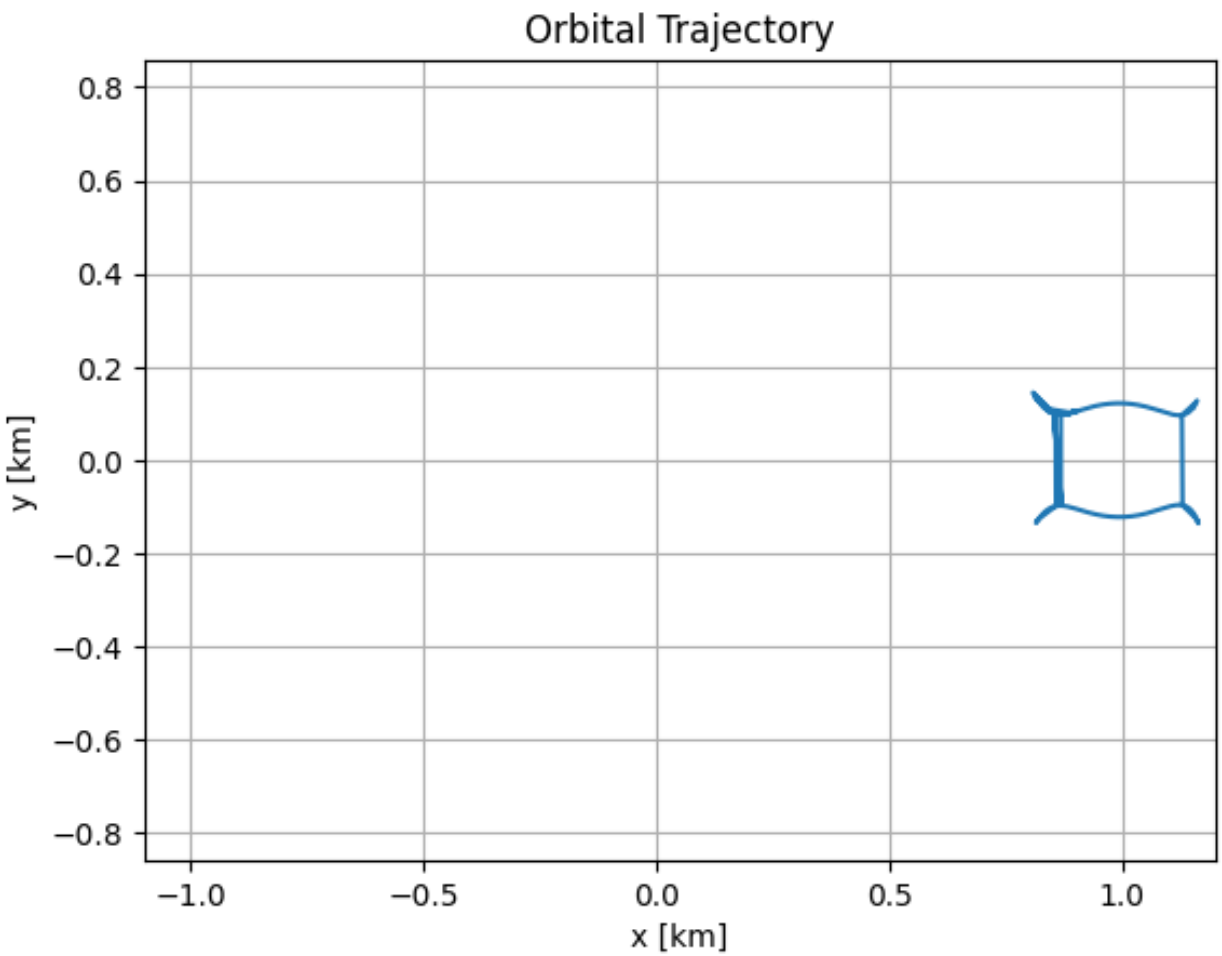

Figure 5: Result of a bad implementation

3 Good 2D Solution

Instead, implement this properly as YAPSS being able to modify the velocity with the control. Additionally, remove the difference of the final velocities as part of the objective; This should be covered by the Jacobi constant and the control minimization. Thus, minimize:

$$\int W_c \left(C - C*\right)^2 + W_u \left(v_x^2 + v_y^2\right) dt \tag{3.1}$$

With both weights at 1, this generates a very good solution. The period error when compared to the paper is 2.59×10^{-6} percent, or about 0.1 seconds. Even with no final constraint on either of the velocity components (though the x velocity is bounded to zero initially and finally to satisfy symmetry), the difference in y velocity is 2.11×10^{-15}. The graphs look good and speak for themselves.

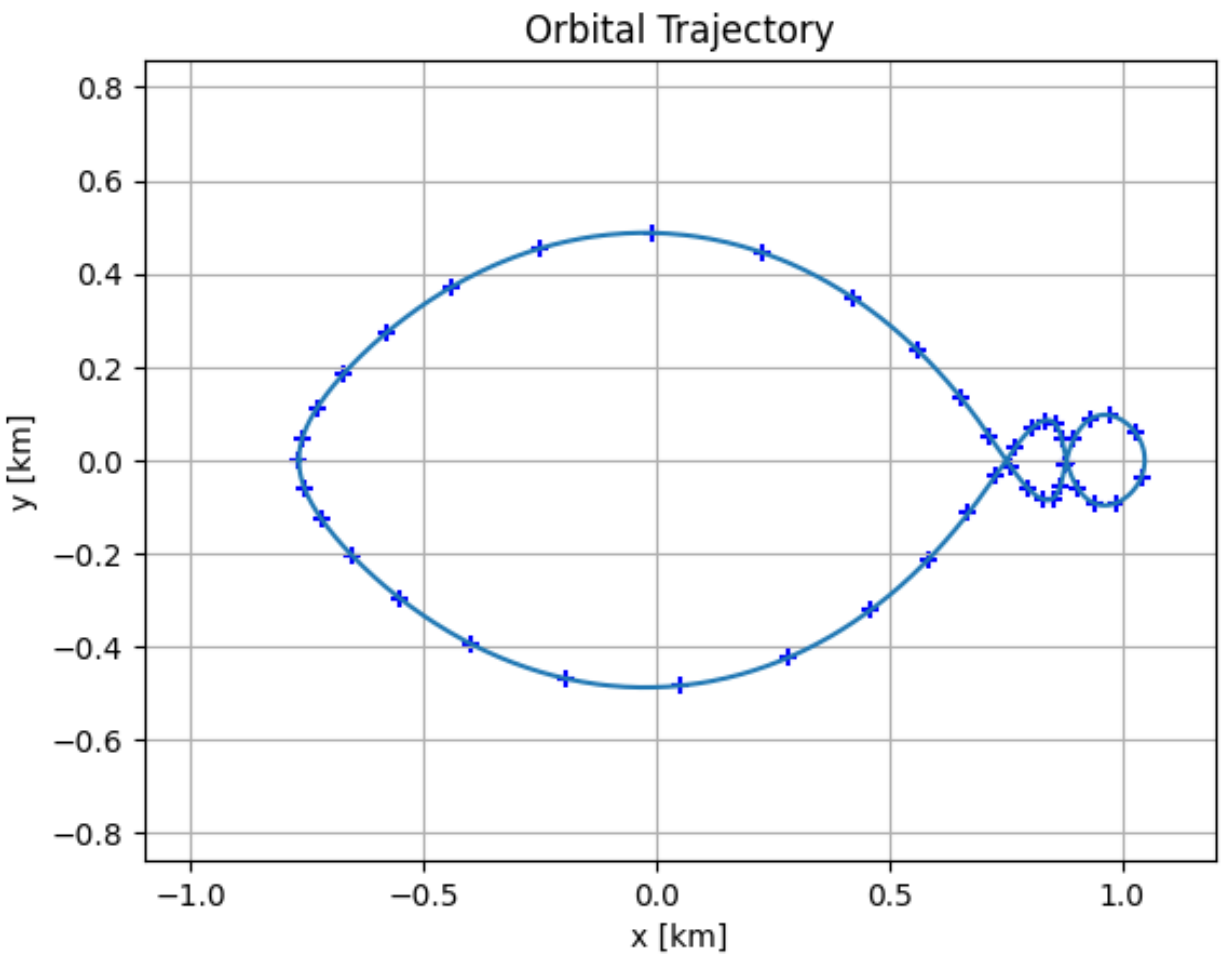

Figure 6: Good 2d orbit

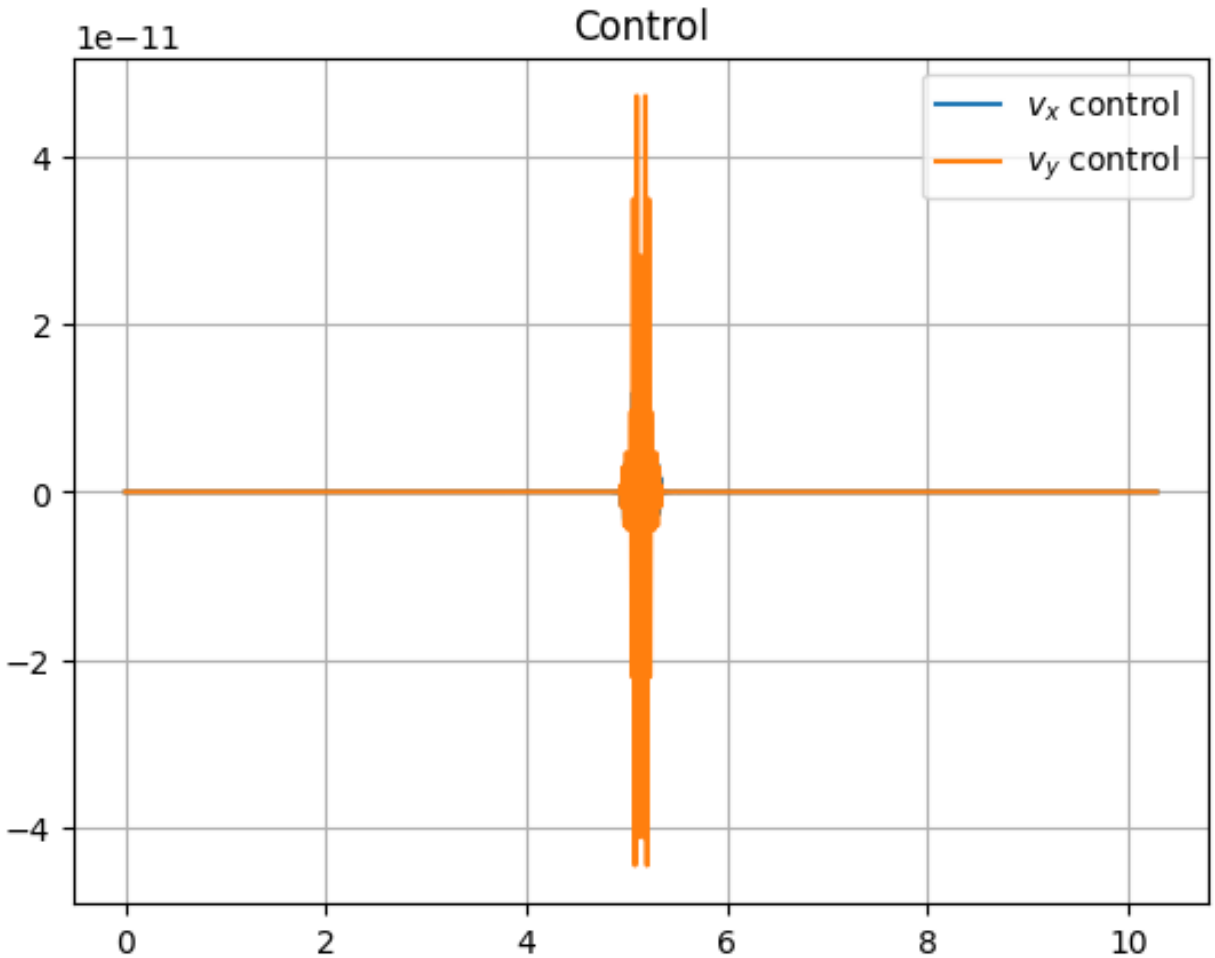

Figure 7: Control for good orbit

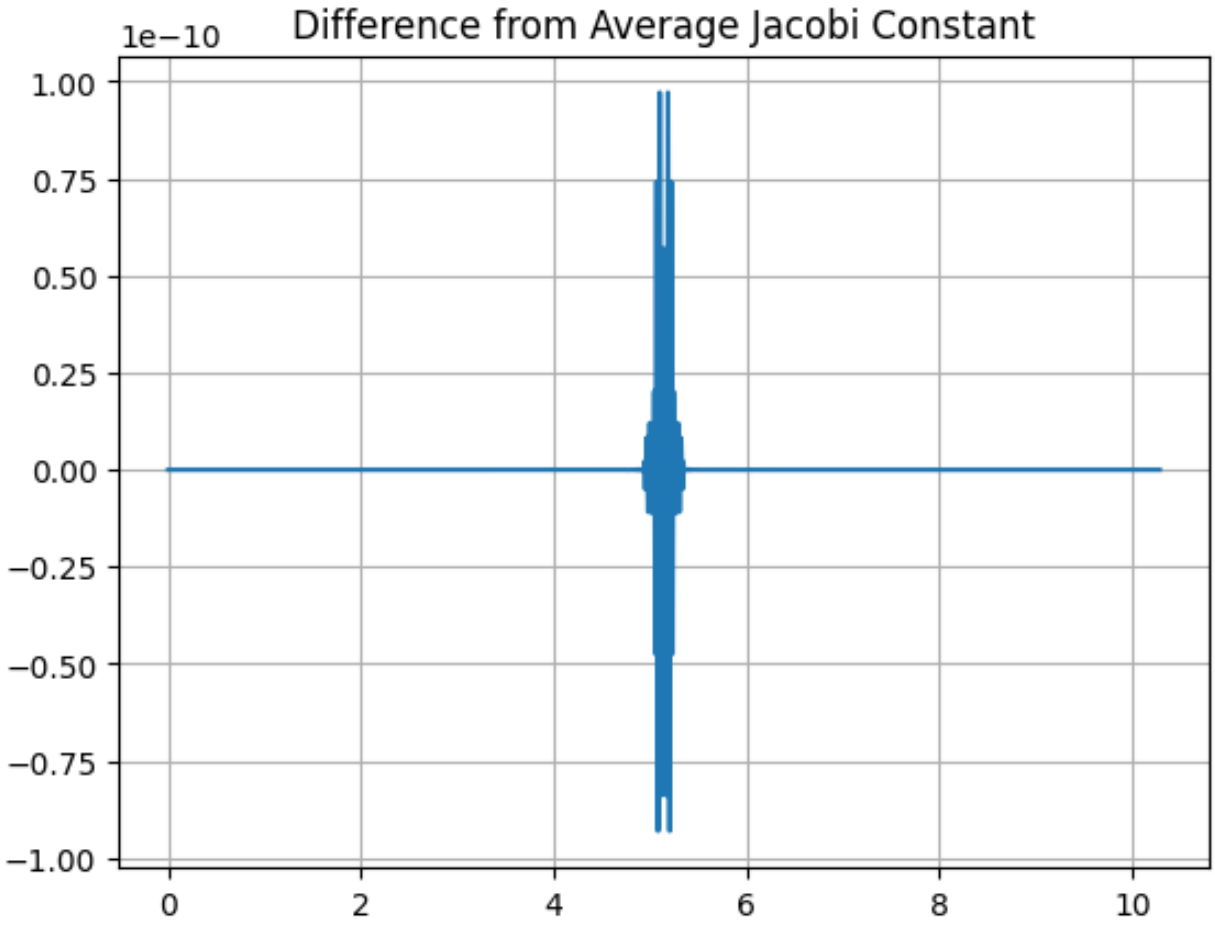

Figure 8: Jacobi difference for good orbit

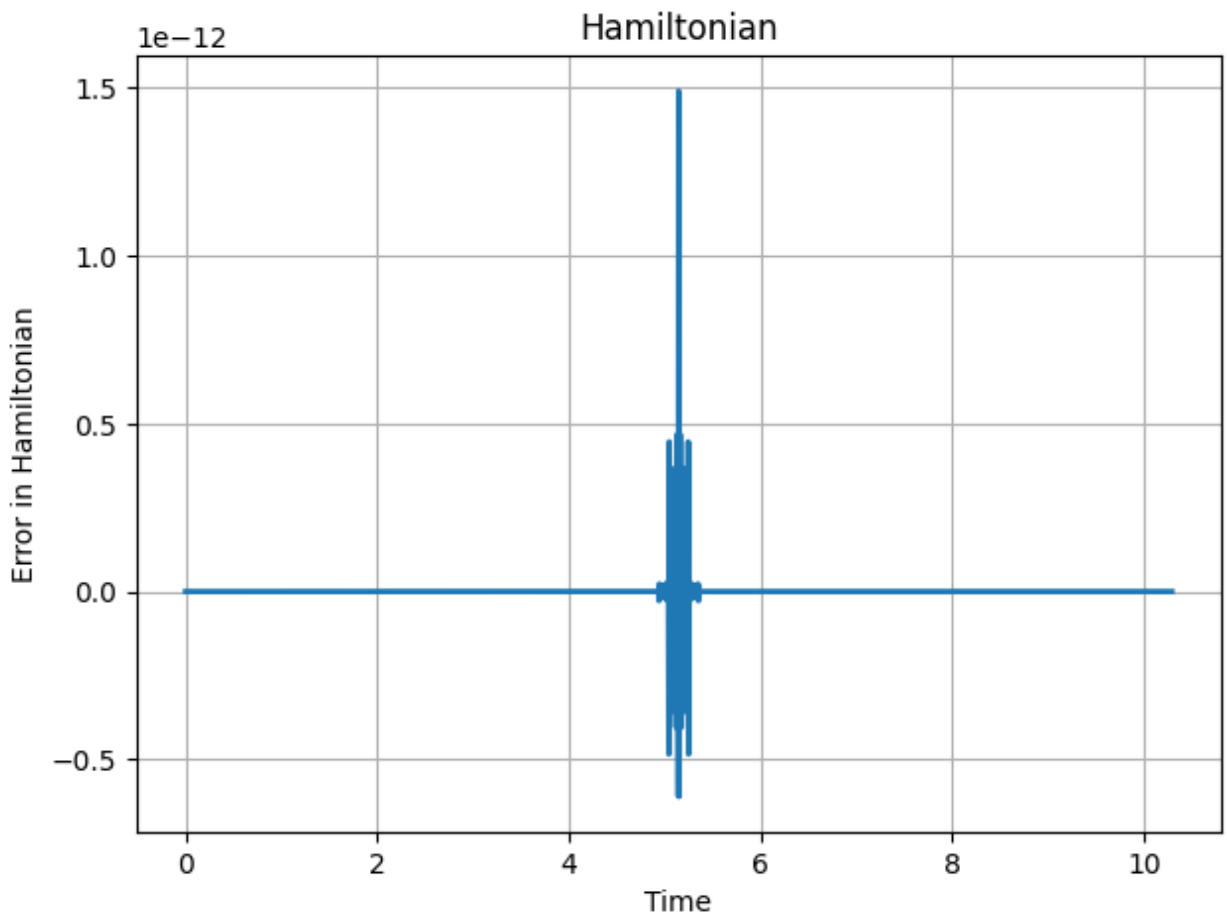

Figure 9: Hamiltonian for good orbit

4 Better 2D Solution

I realized that I was defining my initial guess for the velocities incorrectly, and after fixing that, the solutions found were on the order of 1×10^{-23} time units long. Without a final cost, YAPSS would apply a large amount of control and ignore the Jacobi constant to just keep the spacecraft in place, which was the minimal value on account of the short time step. To solve this, I added the final Jacobi constant difference as a part of the cost. Thus, minimize:

$$W_{cf}\left(C_f - C*_f\right)^2 + \int W_c\left(C - C*\right)^2 + W_u\left(v_x^2 + v_y^2\right) dt \tag{4.1}$$

This works very well. Period error is roughly 160 seconds (0.004 percent). The Jacobi is on the order of 1×10^{-10}, the Hamiltonian is on the order of 1×10^{-12}, and the controls demanded are on the order of 1×10^{-11}. Below is a figure of the plot.

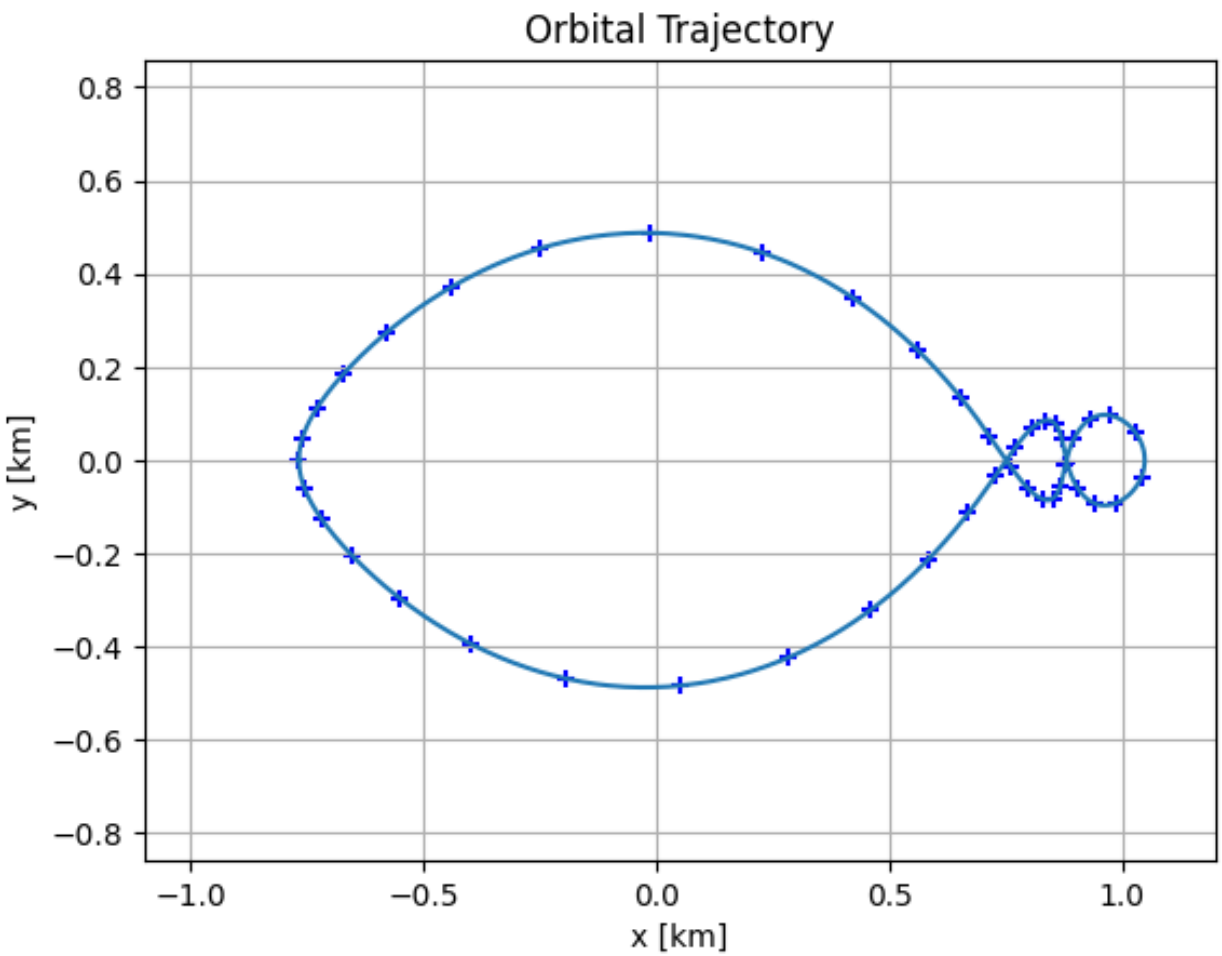

Figure 10: Fixed initial guess and cost functional

5 Extending to 3D

This approach was extended to the 3D model. After adjusting the dynamics and the Jacobi constant equation accordingly, the results roughly match. Note that the final solution for the 2D orbit is used as the initial guess for the 3D orbit. Additionally, while the 2D model takes about 7 seconds to find a solution, the 3D model takes about 3 minutes. The period error dropped to 20 seconds, or 0.0005 percent.

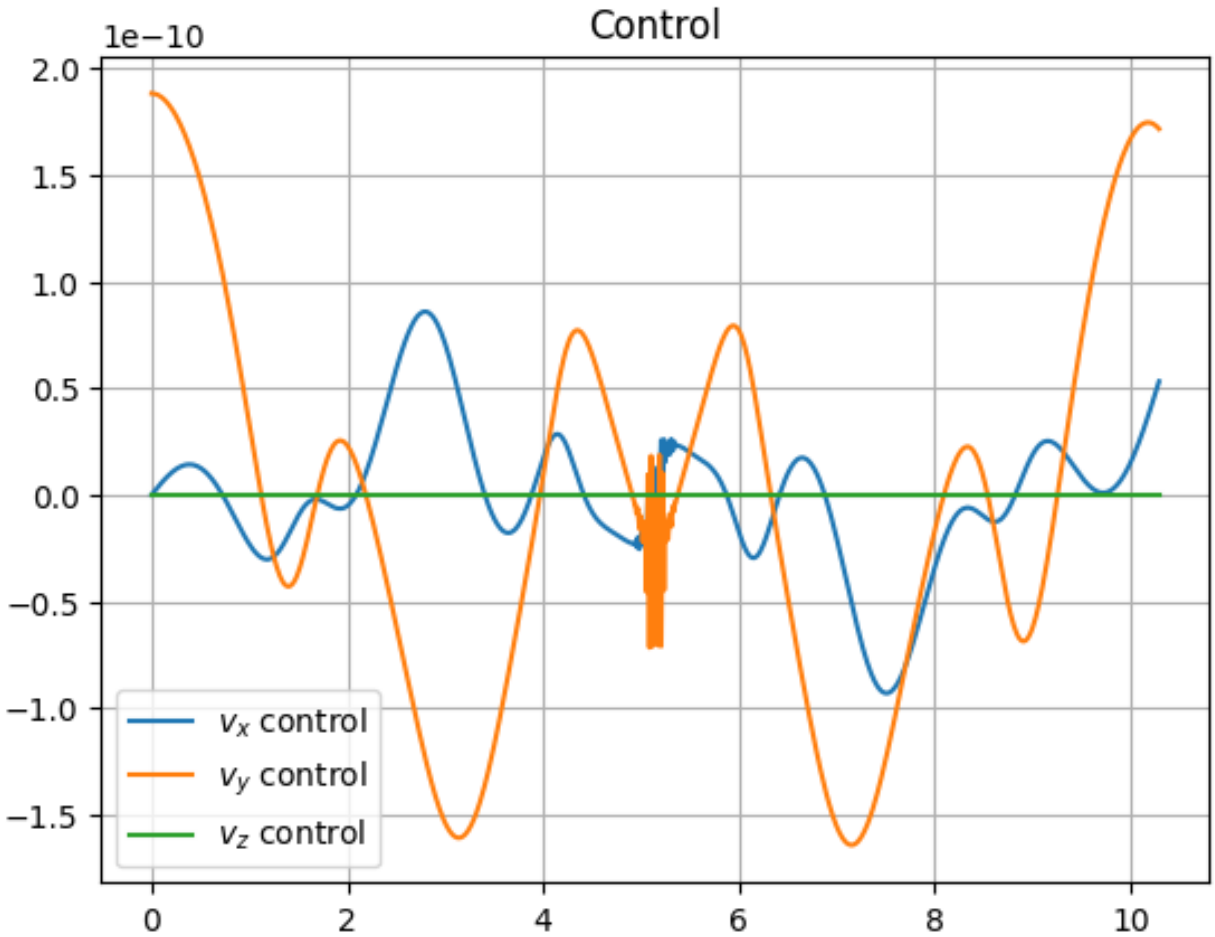

Figure 11: Control for 3D orbit

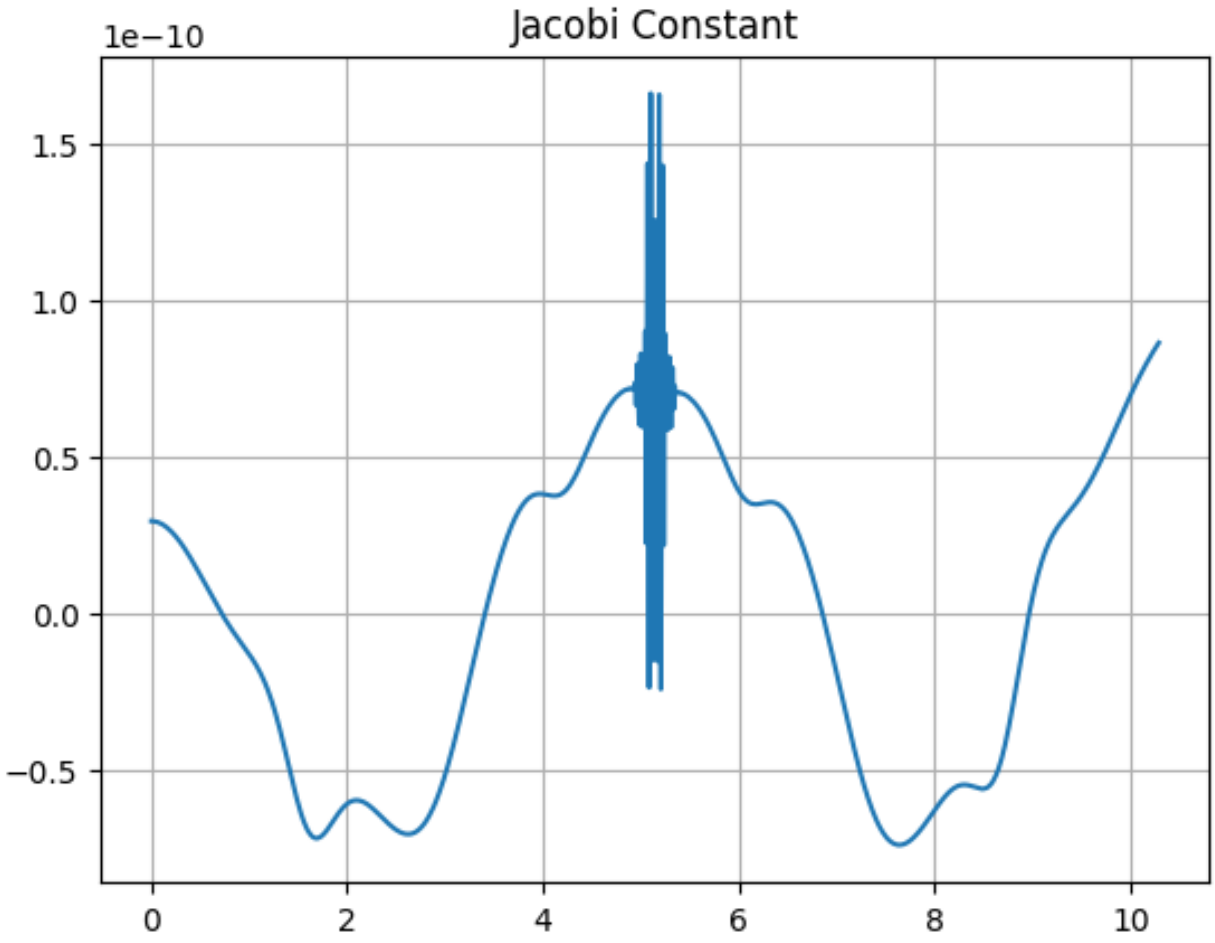

Figure 12: Jacobi for 3D orbit

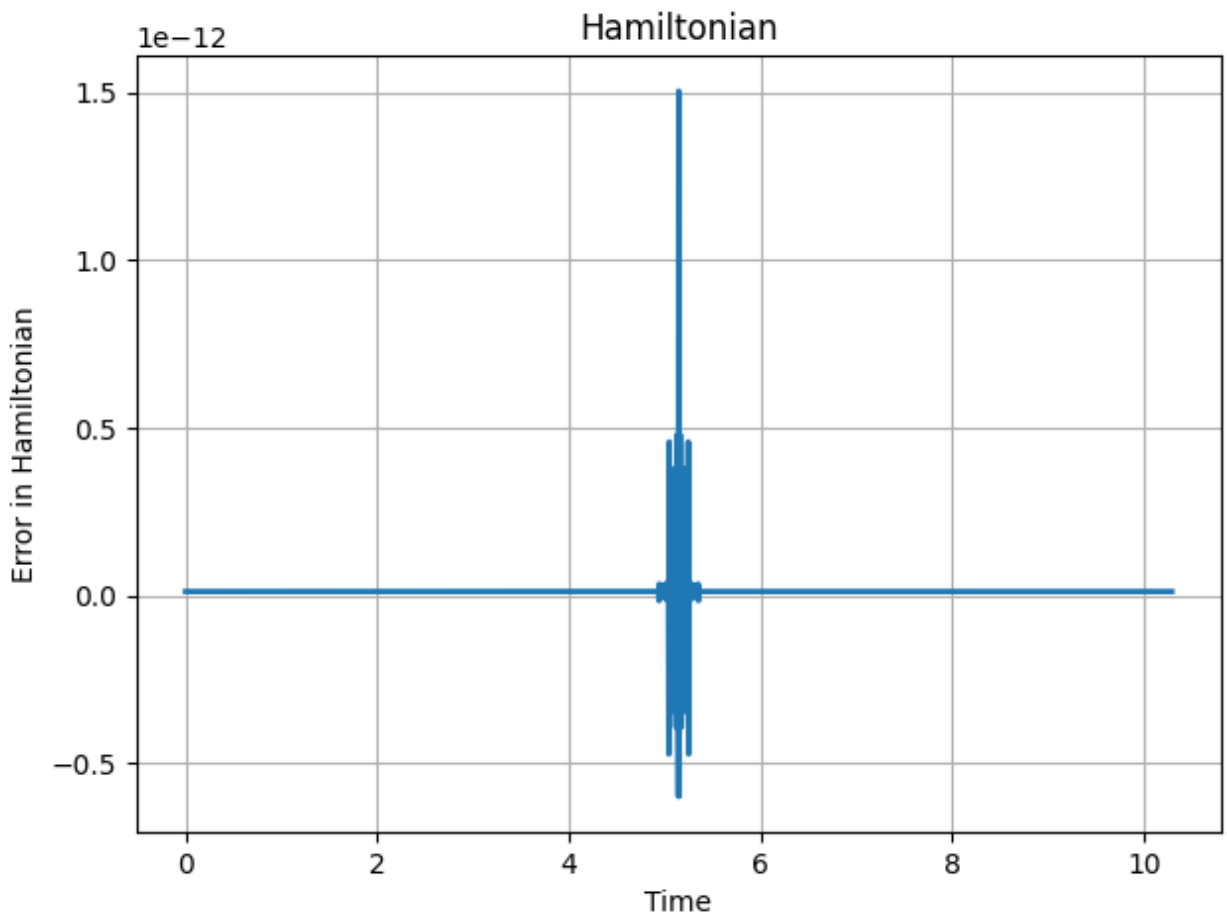

Figure 13: Hamiltonian for 3D orbit